LABA LIBERA ACCADEMIA BELLE ARTI

Corso propedeutico di programmazione

Internet, API, linguaggi di programmazione e web

Andrea Cometti

Indice

1 Internet e reti **1**

1.1 Che cos'è Internet 1

1.2 Percorso di una richiesta 1

1.3 Indirizzi e nomi 1

1.4 Com'è composto un link (URL) 1

2 API e comunicazione tra sistemi **4**

2.1 Che cos'è un'API 4

2.2 Autenticazione e autorizzazione 4

3 Linguaggi di programmazione **4**

3.1 General purpose e specifici di dominio 4

3.2 Livello di astrazione 5

3.3 Compilazione ed esecuzione 5

3.4 REPL 5

1 Internet e reti

Prima di scrivere codice conviene capire dove, di fatto, quel codice vive e come comunica. Quasi ogni programma che useremo, direttamente o indirettamente, si appoggia a Internet: vediamo come funziona, partendo dalle basi.

1.1 Che cos'è Internet

Internet è una **rete di reti**: non è un'unica grande infrastruttura, ma l'insieme di tante reti di dispositivi sparse per il mondo, collegate tra loro in modo da potersi scambiare dati.

Per farlo, Internet usa la **comunicazione a pacchetto**. I dati non viaggiano in un unico blocco: vengono suddivisi in piccole unità, i *pacchetti*, che attraversano la rete e vengono ricomposti o elaborati a destinazione. Può capitare che pacchetti diversi della stessa comunicazione seguano percorsi differenti.

A mettere ordine in tutto questo c'è **TCP/IP**, una *suite* di protocolli, cioè di regole condivise che permettono a dispositivi anche molto diversi di capirsi. IP si occupa dell'indirizzamento e dell'inoltro dei pacchetti; TCP, quando viene utilizzato, aggiunge tra le altre cose una trasmissione affidabile e ordinata.

1.2 Percorso di una richiesta

Quando apriamo una pagina o usiamo un'app, la nostra richiesta compie un viaggio. In forma semplificata possiamo immaginarlo così:

utente → router → Internet → server → risposta all'utente

Si tratta appunto di uno schema semplificato: il percorso reale dipende dalla rete, dal servizio e dai protocolli coinvolti. In questo viaggio compare anche il **gateway**, un punto che collega reti o sistemi differenti. In una rete domestica, questo ruolo è spesso svolto proprio dal router.

1.3 Indirizzi e nomi

Perché una richiesta arrivi a destinazione, serve un modo per indicare *a chi* è destinata. Entrano in gioco tre concetti.

L'**indirizzo IP** è un indirizzo logico che identifica un'interfaccia di rete e permette di instradare i dati. Un dispositivo può avere più interfacce, e quindi anche più indirizzi.

Ricordare sequenze di numeri non è però comodo per le persone, ed è qui che interviene il **DNS**: un sistema che traduce i nomi di dominio, come `example.com`, in informazioni utili a raggiungere i servizi, tra cui gli indirizzi IP. È un po' la rubrica telefonica di Internet.

Infine **localhost**, un nome convenzionale che indica il dispositivo su cui stiamo lavorando. Di solito corrisponde agli indirizzi di loopback `127.0.0.1` (IPv4) e `::1` (IPv6). Attenzione a un equivoco frequente: localhost non è l'indirizzo pubblico del dispositivo, ma un modo per parlare con se stessi.

1.4 Com'è composto un link (URL)

Sappiamo ora che per raggiungere un servizio servono un indirizzo e un nome. Nella pratica, però, quando navighiamo ci basta un solo elemento che riunisce tutto: l'URL.

Un **URL** (*Uniform Resource Locator*) è l'indirizzo che permette di individuare una risorsa su Internet, come una pagina web, un'immagine o un documento.

Per esempio, consideriamo questo URL:

`https://www.esempio.it:8080/prodotti/scarpe?id=25#dettagli`

Un URL può essere suddiviso in diverse parti, che vediamo una alla volta.

1. Protocollo (schema)

`https://`

Indica il protocollo utilizzato per comunicare con il server. I due più comuni sono:

- **HTTP** (*HyperText Transfer Protocol*): protocollo utilizzato per trasferire le informazioni tra browser e server;
- **HTTPS**: versione protetta di HTTP, che utilizza la crittografia tramite TLS per proteggere la comunicazione.

2. Dominio (nome del sito)

`www.esempio.it`

È il nome che identifica il sito o il server a cui vogliamo collegarci. Si può suddividere in:

- `www`: *sottodominio*, tradizionalmente usato per il web, ma non obbligatorio;
- `esempio`: nome del dominio;
- `.it`: *dominio di primo livello* (TLD), che in questo caso identifica il dominio nazionale italiano.

Il dominio permette di raggiungere il server attraverso il sistema DNS, che traduce il nome del dominio nell'indirizzo IP corrispondente.

3. Porta

`:8080`

Indica la porta di comunicazione del server a cui il browser deve collegarsi. Alcune porte comuni sono:

- `80`: porta predefinita per HTTP;
- `443`: porta predefinita per HTTPS.

Quando si utilizza la porta predefinita, generalmente non è necessario scriverla nell'URL.

4. Percorso (path)

```
/prodotti/scarpe
```

Indica il percorso della risorsa richiesta all'interno del sito. In questo esempio, il percorso identifica una risorsa associata a `/prodotti/scarpe`. Non è detto che corrisponda a una vera cartella sul server: può essere gestito da un'applicazione web.

5. Parametri della query string

```
?id=25
```

Sono informazioni aggiuntive inviate al server, spesso utilizzate per filtrare risultati, identificare contenuti o modificare una ricerca.

- `?`: segna l'inizio della query string;
- `id`: nome del parametro;
- `25`: valore del parametro;
- `&`: permette di separare più parametri, per esempio `?id=25&colore=rosso`.

I parametri non sono necessariamente visibili o utilizzati nello stesso modo da tutti i siti.

6. Frammento (fragment o anchor)

```
#dettagli
```

Indica una parte specifica della risorsa, per esempio una sezione di una pagina HTML.

- `#`: introduce il frammento;
- `dettagli`: identifica la sezione o il punto di destinazione.

Il frammento viene normalmente gestito dal browser e non viene inviato al server nella richiesta HTTP.

Schema riassuntivo

Un URL può avere questa struttura generale:

```
schema://dominio:porta/percorso?parametri#frammento
```

Non tutte le parti sono obbligatorie: per esempio, la porta, la query string e il frammento possono mancare.

Da ricordare: l'URL identifica la posizione di una risorsa o il modo per raggiungerla; il **link**, invece, è un collegamento cliccabile che può rimandare a un URL.

2 API e comunicazione tra sistemi

Dopo aver visto come viaggiano i dati, possiamo chiederci come fanno due programmi a chiedersi qualcosa a vicenda. La risposta, molto spesso, sono le API.

2.1 Che cos'è un'API

Un'**API** (*Application Programming Interface*) è un insieme di regole e strumenti con cui un software mette a disposizione funzioni o dati che altri software possono utilizzare. In pratica, un'API definisce *come* richiedere un servizio e *che tipo di risposta* aspettarsi. Alcune API sono locali, altre permettono la comunicazione attraverso una rete.

Un esempio concreto: un'app meteo non possiede le previsioni, ma può usare un'API per chiederle a un server e poi mostrare i risultati all'utente.

2.2 Autenticazione e autorizzazione

Quando un servizio è esposto tramite API, di solito vuole sapere *chi* lo sta usando e *cosa* gli è permesso fare. Sono due domande distinte, e vale la pena tenerle separate.

L'**autenticazione** è la verifica dell'identità di un utente o di un sistema: "sei davvero chi dici di essere?". L'**autorizzazione**, invece, stabilisce quali operazioni o risorse sono consentite a quell'utente o sistema: "cosa puoi fare, una volta riconosciuto?".

Per gestire questi aspetti si incontrano spesso due strumenti dai nomi simili, che però non vanno confusi:

- **API key**: una chiave usata per identificare un'applicazione o un progetto e, a seconda del servizio, controllarne l'accesso e l'utilizzo. Non equivale necessariamente a nome utente e password, e non va considerata sempre sufficiente per autenticare una persona.
- **Passkey**: una credenziale di accesso basata su crittografia a chiave pubblica, usata per autenticare un utente. Nonostante l'assonanza, in generale non è un tipo di API key.

3 Linguaggi di programmazione

Ora che sappiamo come i sistemi comunicano, passiamo a come si scrivono i programmi. I linguaggi di programmazione sono tanti, e possono essere classificati con criteri diversi.

3.1 General purpose e specifici di dominio

Un primo modo per distinguerli è guardare allo *scopo*. I linguaggi **general purpose** si possono usare per molti tipi di problemi e applicazioni: Python, Java, C e C++ ne sono esempi. I linguaggi **specifici di dominio** (*DSL, Domain-Specific Language*) sono invece progettati per un ambito più circoscritto: SQL, per esempio, serve a interrogare e gestire dati nei database, mentre altri linguaggi descrivono circuiti o controllano macchine.

Questa distinzione dipende dallo scopo e dal contesto, non dalla "forza" del linguaggio: un linguaggio specifico non è necessariamente meno potente, è semplicemente pensato per un insieme più limitato di compiti.

3.2 Livello di astrazione

Un altro criterio è il *livello di astrazione*, cioè quanto il linguaggio ci tiene lontani dai dettagli della macchina. I linguaggi **di alto livello** offrono costrutti più vicini al modo in cui le persone descrivono un problema e nascondono molti dettagli tecnici. I linguaggi e le istruzioni **di basso livello**, come il linguaggio macchina e l'assembly, sono invece più vicini all'architettura hardware.

C e C++ meritano una nota a parte: vengono generalmente classificati come linguaggi di alto livello, ma permettono anche un controllo dettagliato della memoria e operazioni molto vicine alla macchina.

A questo punto è utile fissare due termini che si confondono facilmente. Il **codice sorgente** è il testo che scriviamo in un linguaggio di programmazione; il **codice macchina** è la rappresentazione delle istruzioni eseguibili direttamente dal processore. Non sono sinonimi: il primo è pensato per le persone, il secondo per l'hardware.

3.3 Compilazione ed esecuzione

Se il codice sorgente non è eseguibile direttamente dal processore, come fa a diventare un programma in funzione? Ci sono due strategie principali.

La **compilazione** traduce il codice sorgente, o una sua parte, in una rappresentazione eseguibile o intermedia. L'**interpretazione**, invece, prevede un interprete che legge ed esegue le istruzioni, oppure una loro rappresentazione, man mano.

Spesso si sente parlare di linguaggi "compilati" e "interpretati" come di due categorie rigide, ma la distinzione non è assoluta: lo stesso linguaggio può essere eseguito con tecniche diverse. JavaScript, per esempio, può essere interpretato e allo stesso tempo compilato durante l'esecuzione tramite *JIT* (*Just-In-Time*).

Quanto alle **prestazioni**, la compilazione può migliorarle e consentire ottimizzazioni, ma la velocità e l'efficienza dipendono anche dal programma, dall'ambiente e dall'implementazione specifica: non basta sapere se un linguaggio è compilato per dire che sarà più veloce.

3.4 REPL

Quando si impara a programmare è molto utile poter provare un'istruzione e vederne subito l'effetto. Lo permette il **REPL**, acronimo di *Read–Eval–Print Loop*, che funziona in quattro tempi:

1. **Read**: legge un comando;
2. **Eval**: lo valuta o lo esegue;
3. **Print**: mostra il risultato;
4. **Loop**: ripete il ciclo, tornando alla lettura di un nuovo comando.